Research Proposal Draft

Scoped Verification Reuse for Agent-Acquired Coding Experience Across Repository Revisions

Tobias Peeters

October 20, 2026

1 Motivation and Problem

Software engineering agents can store experience from previous tasks and retrieve it during subsequent work. Such experience may include successful strategies, explanations of failed attempts, and observations about repository behaviour. Recent work demonstrates that appropriately selected experience can improve coding-task performance, although its usefulness depends on its representation and relevance [1, 2].

However, repository changes can invalidate assumptions underlying stored lessons. Changes to APIs, configuration, dependencies, or program behaviour may make previously useful advice misleading. Reusing this advice can cause incorrect edits or unnecessary debugging. Conversely, discarding every lesson associated with changed code can remove guidance that remains applicable.

Rechecking every lesson after each revision could be expensive. Moreover, independently acquired lessons may rely on the same underlying assumption, creating opportunities to share verification evidence. We investigate whether these opportunities occur naturally and whether exploiting them can reduce maintenance cost while preserving memory reliability and coding-agent performance.

Illustrative example. Several lessons may recommend checking whether `get_user()` returns `None` for a missing user. A later revision changes this behaviour to raise `UserNotFound`. Where the relevant inputs and environment conditions match, one targeted check could contradict the shared assumption and flag all dependent guidance. More general advice, such as handling missing users explicitly, may remain applicable.

2 Research Idea and Positioning

We propose to connect agent-acquired lessons to explicit, scoped assumptions and supporting evidence. Evidence may include execution results, targeted tests, or code observations, together with the revision and conditions under which they were obtained.

Following a repository change, dependency analysis would identify assumptions that potentially require reassessment. A changed dependency would trigger investigation rather than automatically establish invalidity. Assumptions occurring in only one lesson would remain eligible for checking.

Where multiple lessons rely on the same assumption under compatible conditions, they could share a verification result. Existing evidence could also be reused across revisions when its scope and relevant dependencies justify doing so. Results would distinguish assumptions that are supported within scope, contradicted, or inconclusive. A supported assumption would not, by itself, establish the correctness or usefulness of an entire lesson.

The initial system would record these outcomes and flag or withhold guidance relying on contradicted assumptions while preserving its history. Automatically rewriting or pruning lessons would be considered separately, since detecting a false assumption does not necessarily establish a correct replacement.

Positioning. The proposed contribution is not memory storage, memory updating, or dependency-based checking alone. Related work already addresses experience retrieval, evolving memory, dependency-aware invalidation, and selective verification. Our focus is whether scoped assumptions can be identified across independently acquired coding lessons, when they can legitimately share evidence, and whether this provides benefits beyond dependency tracking and exact-check caching. The precise contribution will depend on the pilot findings and comparison with the closest maintenance methods.

Dependency-based checking selects checks whose recorded code or environment dependencies have changed. Exact-check caching avoids repeating an identical check when its relevant inputs and execution conditions remain unchanged. These mechanisms form the baseline against which we will assess additional savings from sharing evidence across distinct lessons.

Optional memory categorization. We may additionally investigate whether distinguishing general procedural guidance from concrete repository-dependent claims improves verification routing. This classification would be treated as an exploratory signal, not a sufficient reason to declare a lesson valid or exempt it from checking. Lessons containing both general and specific content would be assessed at the claim level. Categorization would be included in the final method only if it provides measurable value beyond assumption and dependency information.

3 Research Questions

1. Do naturally acquired coding lessons contain checkable, repository-dependent assumptions whose applicability changes across revisions?
2. Under what conditions can evidence be shared across independently acquired lessons or reused across revisions, beyond what dependency tracking and exact-check caching already provide?
3. Can this reduce total maintenance cost while preserving memory reliability and coding-agent performance?

4 Initial Pilot Plan

We provisionally use a small subset of SWE-ContextBench, beginning with approximately 10–20 base tasks from a few Python repositories. Repository history will supplement the task metadata. Chronological relationships and reproducible environments will be checked before selecting revision sequences. EvoArena remains a possible additional source of evolution sequences rather than a prerequisite for the initial pilot.

Pilot A: Experience acquisition and source validity. A fixed agent and backbone model will attempt the selected base tasks. We will record trajectories, source commits, generated patches, relevant workspace states, and execution environments. Lessons from successful and unsuccessful steps will be inspected for scoped assumptions and supporting evidence.

We will distinguish assumptions supported by the historical repository from those depending exclusively on the agent's unmerged edits. Manual review will assess how often independently acquired lessons share assumptions, without treating surface similarity as evidence of equivalence.

Pilot B: Applicability across revisions. We will assess source-supported assumptions at later revisions, including both behaviour-changing edits and changes that leave the assumptions supported. Independent assessment will label assumptions as supported within scope, contradicted, or inconclusive.

Alongside the original memory bank, controlled variants will replace selected lessons with constructed advice whose assumptions held at the source revision but no longer hold at the target revision. Matched valid lessons will provide controls for unnecessary rejection. Natural and constructed cases will be reported separately, and evaluation labels will remain unavailable to the maintenance mechanism.

Pilot C: Verification cost and evidence sharing. We will compare no maintenance, exhaustive checking, dependency-based selective checking, selective checking with exact-result caching, and scoped verification reuse. The comparison will distinguish lesson counts, claim occurrences, distinct scoped claims, and actual check executions.

Manually reviewed mappings may first estimate the available reuse opportunity; results using such mappings will be distinguished from automatic grouping. We will measure verification coverage, incorrect reuse, withholding of supported guidance, and total cost, including extraction, dependency tracking, grouping, and checking.

Memory-type categorization may be assessed as an optional extension. Its contribution will be isolated by comparing otherwise identical routing mechanisms with and without the category information.

Pilot D: Effects on subsequent tasks. Where suitable later tasks exist, we will compare unmaintained memory, dependency-based maintenance with exact-check caching, and the proposed method. No-memory and experimentally corrected memory conditions will serve as diagnostic controls.

Selected lessons may initially be supplied directly to isolate their effects, followed by experiments including retrieval. We will measure task success, regressions, token usage, and execution time, including maintenance overhead. Repeated runs will be used where feasible.

Progression. Pilots A and B will establish the available assumptions, revision changes, and sharing opportunities before substantial implementation. Pilots C and D will proceed where these findings justify further evaluation. Exclusions and inconclusive cases will be reported.

5 Expected Outcomes and Open Questions

The pilot should establish whether scoped verification reuse addresses a sufficiently common and tractable problem in naturally acquired coding experience. Expected outputs include a small annotated collection of lessons, assumptions, supporting evidence, and revision transitions, together with an assessment of the available reuse opportunities and their implementation cost. These outputs remain useful if the proposed mechanism provides limited benefit.

The intended evaluation outcome is lower total maintenance cost than dependency-based checking with exact-result caching, at comparable verification coverage and incorrect-reuse rates, without a practically unacceptable reduction in coding-task performance. Quantitative thresholds and acceptable performance margins will be defined using pilot findings and fixed before held-out evaluation. Reducing cost by checking less or withholding more useful memory will not alone constitute success.

Key open questions concern the granularity of checkable assumptions, the adequacy of supporting evidence, and the conditions under which evidence transfers between lessons or revisions. We must also determine how to handle incomplete dependencies, mixed-content lessons, and inconclusive checks, and whether automatic assumption grouping costs more than the verification it avoids.

Memory-type categorization will remain optional unless it improves the measured cost–reliability trade-off. Automatic lesson revision and pruning may form a possible later extension. The initial focus will remain on assessing applicability, recording uncertainty, and establishing justified evidence reuse.

References

[1] Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. Structurally aligned subtask-level memory for software engineering agents. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 111510–111521. PMLR, 2026.

[2] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A benchmark for context learning in coding, 2026. Version 3.